

StreamScout ? Database Design

ER modeling (conceptual design)

Entity sets (4+ meaningful entities)

- ****Movie****: a movie/series title from the dataset (``content_id``, title metadata, scores)
- ****Platform****: OTT platform (Netflix / Prime Video / Hotstar)
- ****AppUser****: authenticated user for watchlists & reviews
- ****Review****: user's review (stars + optional text) on a movie
- ****Watchlist****: user's saved items with a status

Relationships

- ****Availability**** (Movie ? Platform): many-to-many
A movie can be on multiple platforms and each platform has many movies.
- ****Writes**** (AppUser ? Review ? Movie): one user writes many reviews; each review references one movie.
- ****Saves**** (AppUser ? Watchlist ? Movie): one user has many watchlist rows; each row references one movie.

Preliminary ER diagram (text)

```
...
AppUser (id, username, password_hash, role, ...)
|1
|      Watchlist (id, user_id, movie_id, status, created_at)
|N          |N
|            |1
+-----> Movie (id, content_id, title, type, genre, year, scores, ...)
|            |N
|            |
|            |      Availability (movie_id, platform_id)
|            |N          |N
|            +-----> Platform (id, name)
|
|      Review (id, user_id, movie_id, stars, text, created_at)
|N          |N
+-----> Movie
...
```

ER-to-table conversion (logical schema)

Applying standard ER?relational conversion:

- Each entity becomes a table: ``movie``, ``platform``, ``app_user``, ``review``, ``watchlist``
- Many-to-many relationship becomes a junction table: ``availability(movie_id, platform_id)``
with a composite PK
- 1-to-many relationships become FKs on the ?many? side: ``review.user_id``, ``review.movie_id``, ``watchlist.user_id``, ``watchlist.movie_id``

See implementation in ``db/schema.sql`` (keys, checks, uniqueness, and indexes included).

Normalization (wide table ? 3NF/BCNF)

Start with a wide table

From the CSV, we can imagine a single wide relation:

```
`OTT_ROW(content_id, title, type, genre, platform, country, language, release_year,
duration_minutes, rating, votes, weighted_rating, engagement_score, popularity_score,
trending_score, tags, description, poster_url)`
```

Functional dependencies (FDs)

From the dataset semantics:

- $\backslash(\text{content_id} \rightarrow \backslash)$ all movie attributes (title, type, genre, country, language, release_year, duration_minutes, rating, votes, weighted_rating, engagement_score, popularity_score, trending_score, tags, description, poster_url)
- $\backslash(\text{platform} \rightarrow \backslash)$ is a platform name (small domain), not dependent on content_id in the real world; rather, availability is a relationship:
 - Movie ? Platform (many-to-many in general)

Decomposition

To reach 3NF/BCNF:

- ``Movie(content_id, title, type, genre, country, language, release_year, duration_minutes, rating, votes, weighted_rating, engagement_score, popularity_score, trending_score, tags, description, poster_url)``
- ``Platform(name)``
- ``Availability(content_id, platform_name)`` representing the relationship between movies and platforms

Additional application tables:

- ``AppUser(username, password_hash, role, enabled, created_at)``
- ``Watchlist(user, movie, status, created_at)`` with uniqueness (user, movie)
- ``Review(user, movie, stars, text, created_at)``

ER vs normalization result

Both methods converge on the same core design:

- entity tables for movies and platforms
- a junction table for availability

The physical schema uses surrogate numeric PKs (``id``) for performance and simpler joins, while preserving dataset identity via ``movie.content_id`` with a UNIQUE constraint.

Indexing (required)

Two required indexes (and one extra) are included in ``db/schema.sql``:

- ``idx_movie_title_year`` on ``movie(title, release_year)`` to speed browse/search
- ``idx_availability_platform_movie`` on ``availability(platform_id, movie_id)`` to speed platform joins/aggregations
- ``idx_review_movie_created`` to speed movie review listing and aggregates